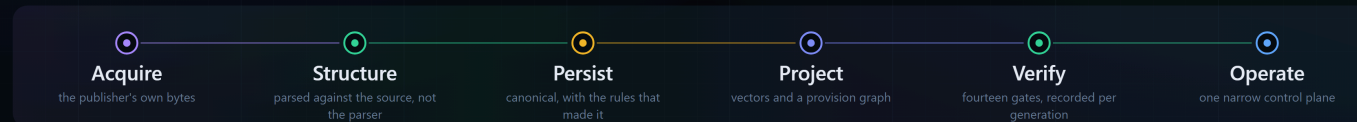


AI Engineering Systems

Mohd Zamin Quadri — AI Engineer

AI Engineering Systems

Legal knowledge · document intelligence · retrieval · verification · event-driven architecture



ONE WRITER EACH

PostgreSQL
canonical record

Qdrant
semantic retrieval

Neo4j
reference traversal

MinIO
immutable evidence

Every stored item re-derivable from the publisher's own bytes, under a recorded set of rules.
When it cannot be, the system says so rather than serving it quietly.

- Mohd Zamin Quadri — AI Engineer, BP-ITCS
- Legal knowledge infrastructure · document intelligence · retrieval · verification
- Four of five services implemented; three more extended

What was actually hard about it?

From legal source to trusted machine knowledge

- Authoritative law changes without telling you, in two unrelated XML markup families
- Extract it for retrieval and you hold three representations that can each drift from the source
- So the problem was provenance and operational control — not retrieval

How does it fit together?

End-to-end legal knowledge architecture

LEVEL 3 · ENGINEERING

Legal Knowledge Database — full architecture

Exact services, topics, routes and stores. One writer per store is an enforced rule, not a convention: the arrows marked exclusive write are the only writes that exist.

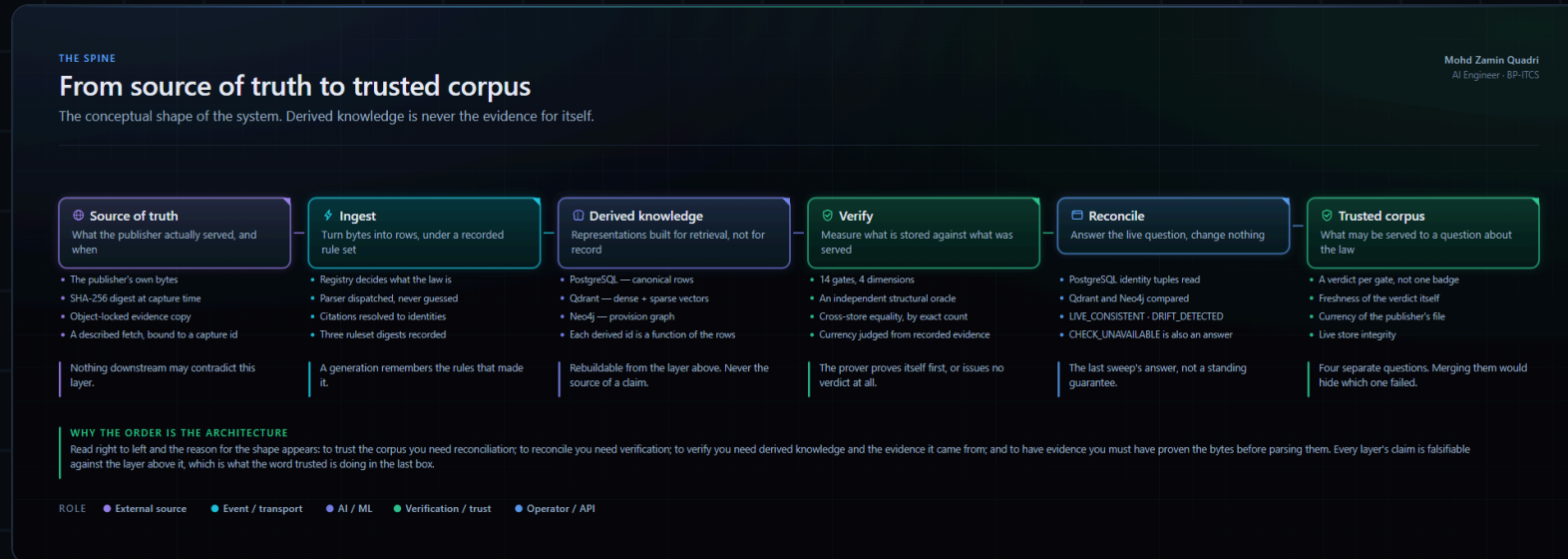
Mohd Zamin Quadri
AI Engineer · BP-ITCS



- Five services: a three-stage ingestion pipeline, plus a two-part control plane
- Three Kafka topics; one law maps to one partition, so generations cannot race
- Exactly one service writes each store — enforced by deleting the path that violated it

What happens to one law?

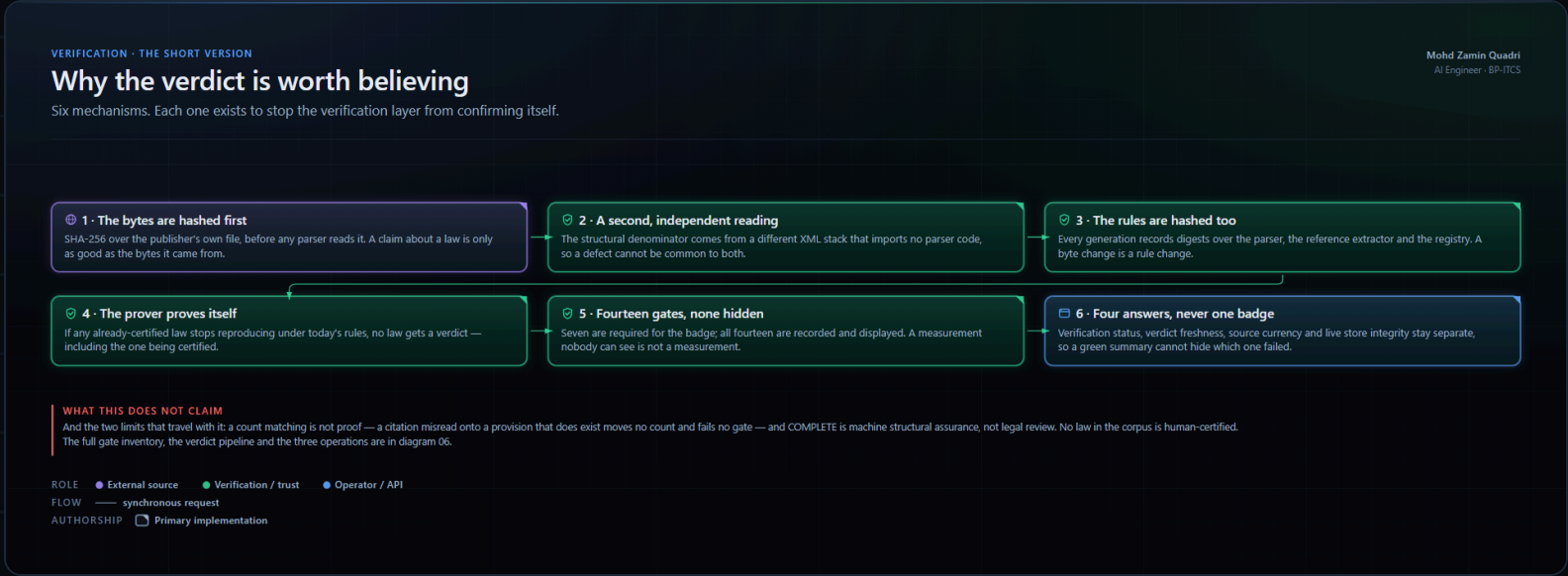
Source → trusted corpus



- Bytes are hashed before anything interprets them
- The structural denominator is built independently of the parser
- law.structured means PostgreSQL only; law.embedded is the completion boundary

How do you know it is right?

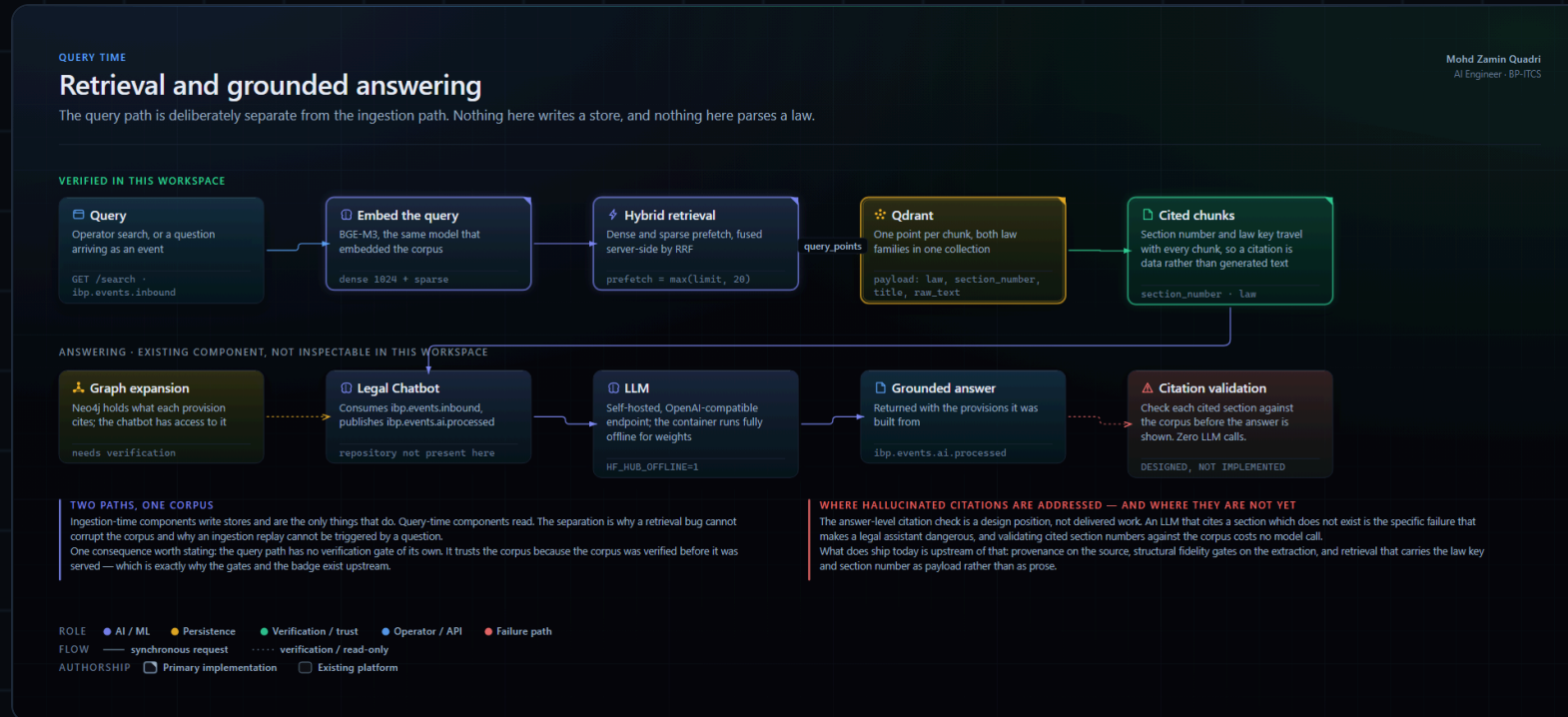
Verification and reconciliation



- 14 gates; 7 required for the badge, all 14 recorded and shown
- The prover proves itself first, or issues no verdict for any law
- A count matching is not proof — and the diagram says so

How is a question answered?

The query path



- BGE-M3 dense + sparse in one pass, fused server-side by RRF
- Citations travel as payload, not as generated prose
- The loader refuses a whole law rather than store a truncated embedding

What happens when it breaks?

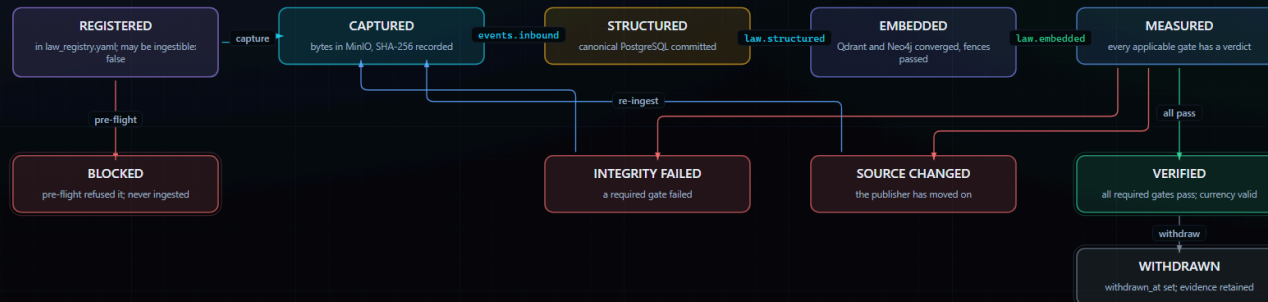
Failure, recovery and operational control

STATE MODEL

The states a law actually occupies

Reconstructed from the technical states, event boundaries, gate verdicts and lifecycle columns that exist in code — not from a generic lifecycle template.

Mohd Zamin Quadri
AI Engineer · BP-ITCS



WHERE THESE STATES LIVE

Two of these are not derived from a status column, and that is the interesting part. EMBEDDED exists because the loader publishes an event only after both count fences pass, and MEASURED exists because the preprocessor consumes that event and runs the gates unasked. So the state model is enforced by the event contract rather than by a field somebody has to remember to set.

WITHDRAWN IS NOT DELETED

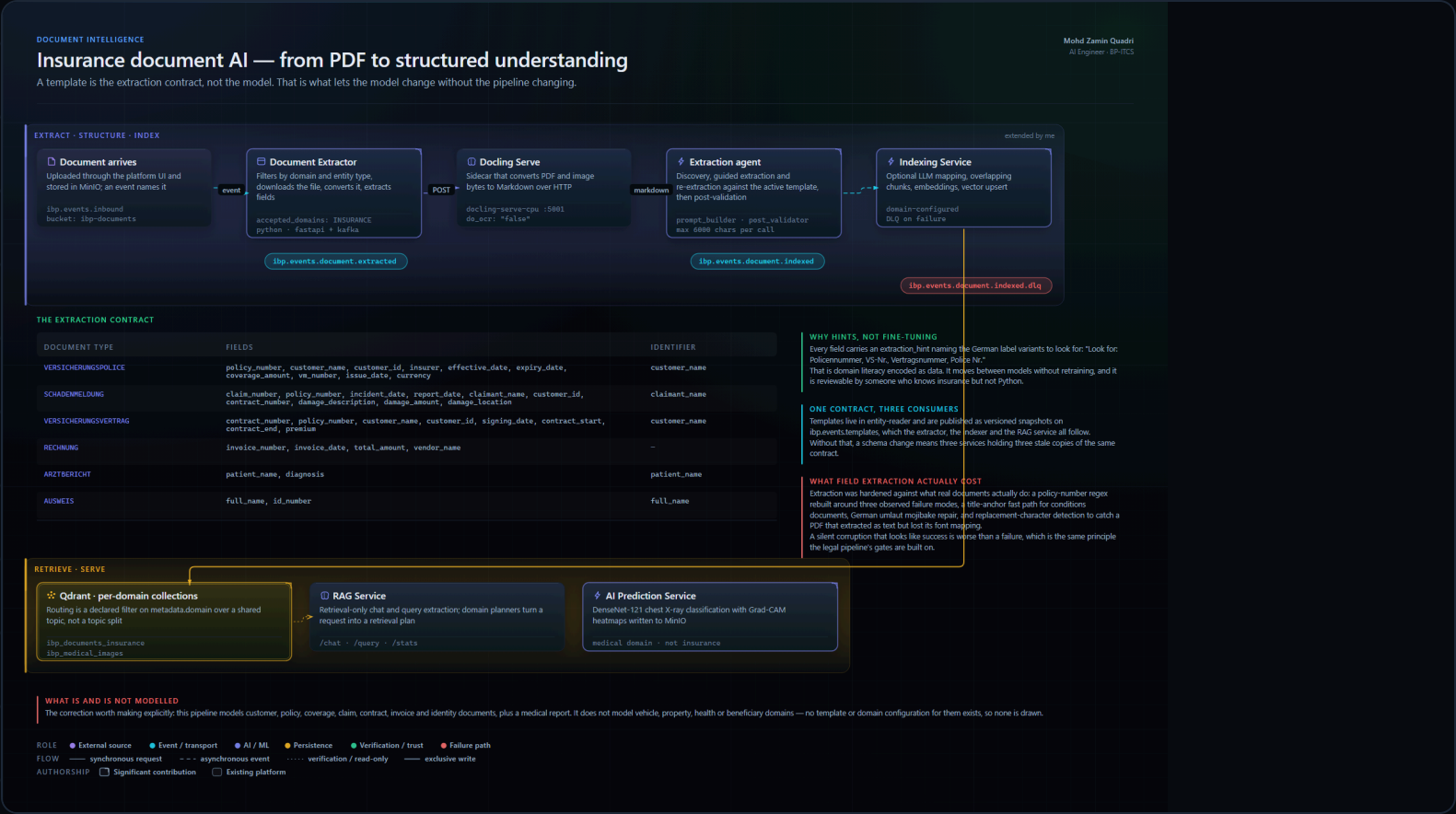
WITHDRAWN is not deletion. The law row survives, withdrawn_at is set, and every piece of evidence — MinIO objects, raw documents, publisher captures, currency observations, verification checks — is untouched. A law leaving the corpus does not make the record of what its publisher served untrue. That sentence is in the source, and it is why withdrawal and deletion are different operations.

ROLE ● External source ● Event / transport ● AI / ML ● Persistence ● Verification / trust ● Operator / API ● Failure path
FLOW — synchronous request - - - asynchronous event — refused / blocked

- Fifteen documented failures, each with its recovery — or the statement that none exists
- Redelivery is safe because identity is derived, never generated
- No dead-letter topic on the legal pipeline: stated as a gap, not hidden

What else runs on the platform?

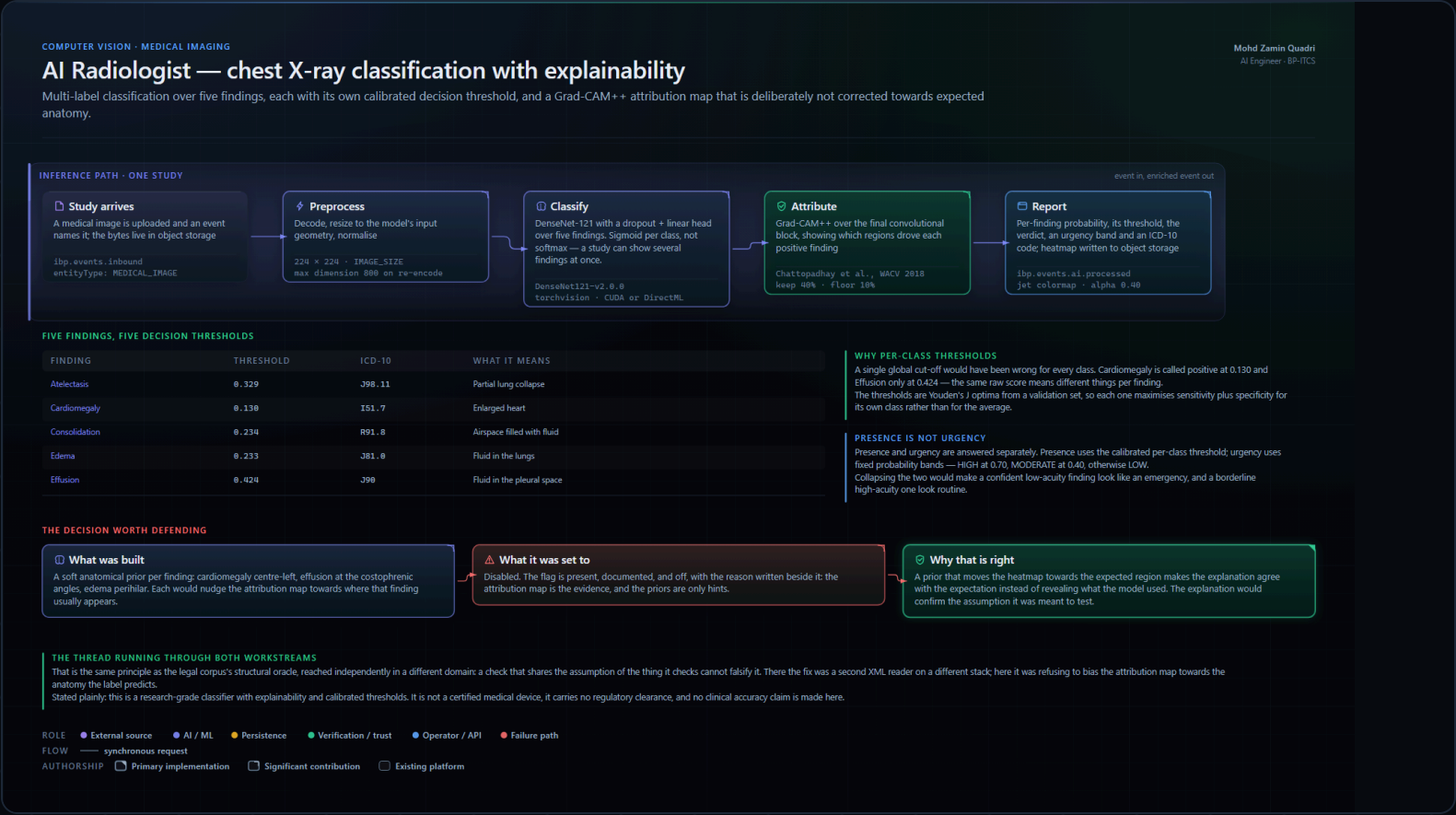
Document intelligence



Is there computer vision too?

Chest X-ray classification

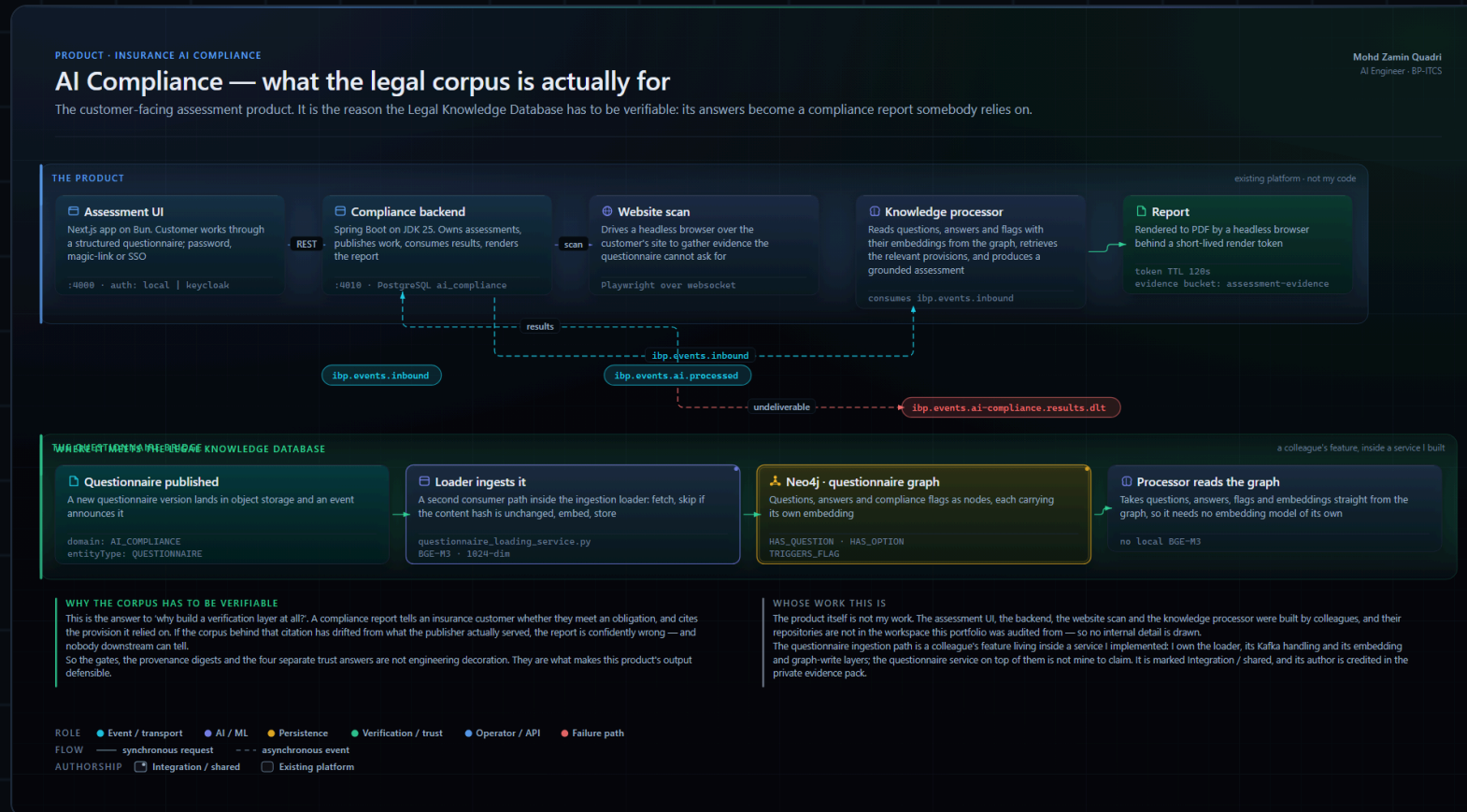
— a research prototype



- Internal project name “AI Radiologist” — not a medical device, not clinically deployed
- DenseNet-121 over five findings, sigmoid per class — a study can show several at once
- One calibrated threshold per finding (Youden's J), plus ICD-10 and an urgency band
- Grad-CAM++ attribution, with anatomical priors built and then deliberately switched off

What is all of it actually for?

AI Compliance — the product the corpus serves



- A questionnaire plus a headless-browser site scan become a grounded, cited compliance report
- A report that cites a drifted provision is confidently wrong — which is why the corpus is verified
- The product is a colleague's work; I built the corpus and the service the questionnaire path runs in

What did you personally build?

What I built, and what I did not

CONTRIBUTION MAP

What I built, what I contributed to, and what I did not

Mohd Zamin Quadri
AI Engineer · BP-ITCS

Five classes, applied to all 20 inspected repositories. Commit share is evidence of weight, not of authorship — where the two disagree, the reason is stated and the conservative class wins.

THE FIVE CLASSES

Primary implementation
I wrote the service and its design docs; majority of commits.

Significant contribution
I added features in a service someone else built, or that I started.

Integration / shared
I wired, configured or contributed to; not mine to claim.

Existing platform
Built by colleagues or third parties; my work depends on it.

External / upstream
Outside the system entirely.

EVERY REPOSITORY, CLASSIFIED

REPOSITORY	MY COMMITS	CLASS	WHAT THE CLASS RESTS ON
knowledge-db-ingestion-preprocessor	406 / 498	Primary	two parsers, 14 gates, oracle, prover, provenance, 84 migrations
legal-hb-dashboard	104 / 167	Primary	7 pages, 27 components, 9 route handlers, the four-ais TrustPanel
legal-hb-health-service	123 / 348	Primary	23 routes, completion consumer, reconciliation, withdrawal orchestrator
knowledge-db-ingestion-loader	77 / 94	Primary	BGE-M3 embedding, hybrid retrieval, dual-store convergence fences
local-rnv	63 / 113	Significant	the whole legal-knowledge-database profile, WAL, archiving, object lock
entity-producer	46 / 116	Significant	EUCCELLAR Claim-Check ingestion, broker-confirmed publish, law-identity keying
document-extractor	27 / 63	Significant	dual LLM-reranks extraction, multi-format input, mjolnir detection
ai-utils	17 / 61	Significant	splitish on Biopython, consumer commit, configurable poll interval
ai-prediction-service	14 / 61	Significant	Grad-CAM analyzer, DemoNetG2 integration, calibrated severity thresholds
architecture	10 / 13	Significant	arc42 documentation, ADR-003 and ADR-004
indexing-service	9 / 39	Significant	I created the service; others continued it. Commit share understates it.
ai-contract-data	7 / 9	Significant	VVG-compliant synthetic document set + generator. Data, not a service.
glaux-ui	5 / 243	Integration / shared	minor changes only
entity-handler	5 / 46	Integration / shared	minor changes only
utils	5 / 189	Integration / shared	minor changes only
entity-reader	1 / 47	Existing platform	consumed for templates
entity-ui-handler	1 / 29	Existing platform	not my work
rag-service	0 / 56	Existing platform	zero commits. Reads what my upstream services produce.
parent	0 / 43	Existing platform	shared Maven parent
base-images	0 / 18	Existing platform	shared container bases

WHAT THAT WORK ACTUALLY WAS

Verification / reliability

Data architecture

AI & document intelligence

Event-driven & operations

WHAT I DID NOT BUILD

Not mine, and stated so wherever this system is described: the Java entity platform, the RAG service (zero commits), the GlauX UI, the legal chatbot, and every third-party store. No production deployment is claimed — only Docker Compose stacks and CI image builds are evidenced. No business impact metric is claimed, because none exists in any repository.

The single sentence: I implemented four of the five services in the Legal Knowledge Database — the system that turns published German and EU law into verifiable machine-readable knowledge, where every store has exactly one writer and the system refuses to certify a law it cannot independently reproduce from the publisher's own bytes.

AUTHORSHIP

Primary implementation

Significant contribution

Integration / shared

Existing platform

External / upstream

- Primary: preprocessor, loader, health service, dashboard — four of the system's five services
- Significant: entity producer, document extractor, indexing service (which I created), ai-utils
- Not mine: the Java entity platform, the RAG service, the chatbot, every third-party store

Mohd Zamin Quadri · AI Engineer · BP-ITCS — every claim traceable to a file | full resolution: assets/png/12-my-contributions.png

11

What would you do differently?

Engineering lessons and design principles

INTERVIEW VIEW - LEVEL 2

From authoritative source to trusted, queryable knowledge

Seven stages. The green-bordered components are the four services I implemented; everything else is marked for what it actually is.

Mohd Zamin Quadri
AI Engineer · BP-ITCS



WHAT IS MINE, AND WHAT IS NOT

Implemented by me

Preprocessor · Loader · Health Service · Dashboard — four of the system's five services

Significant contribution

Entity Producer — it predates legal ingestion; I added EU/CELLAR capture and broker-confirmed publication

Existing platform

PostgreSQL, Qdrant, Neo4j, MiniIO, Kafka, the legal chatbot — integrated and configured, not authored

External / upstream

The publishers themselves. Outside the system entirely.

WHAT THE WORK ACTUALLY WAS

This was not a chatbot over documents. The engineering problem was turning changing, heterogeneous authoritative information into structured, searchable, verifiable machine knowledge while preserving provenance — and giving operators control over ingestion, validation and lifecycle state. Most of the difficulty was in the second half of that sentence.

One rule makes it debuggable: four stores, one writer each. That is why "which service made this row wrong?" always has exactly one answer.

AUTHORSHIP ☒ Primary implementation ☒ Significant contribution ☐ Existing platform ☐ External / upstream

- Keep: one writer per store; derive identity; refuse rather than approximate
- Change: add a dead-letter path; generate the gate list once; content-address the captures
- The lesson: a measurement nobody can falsify is not a measurement